

Tutorial 2: Multidimensional Splines

Advanced Regression

2026-07-30

Two-dimensional splines

For this tutorial, we will use the same example as used in the notes. The code has been adapted from the help file for the `gam` function (library `mgcv`). `mgcv` is a package for fitting GAM models, written by Simon Wood, the expert on all things GAM and thin-plate splines. The description of the package says: *Mixed GAM Computation Vehicle with Automatic Smoothness Estimation*.

GAM models are generalized linear models with spline terms. We will talk about those later, but for the moment we will use the GAM machinery to fit simple splines.

The main fitting function is `gam()` and it works exactly like `glm()`. To add smooth terms to `gam()` use the `s(x)` function. `s(x)` fits thin-plate splines by default.

The `gamSim(2)` function can be used to simulate observations from a surface, adding noise to the true underlying surface. 2 refers to example 2. `scale` is for the level of noise.

```
library(mgcv)

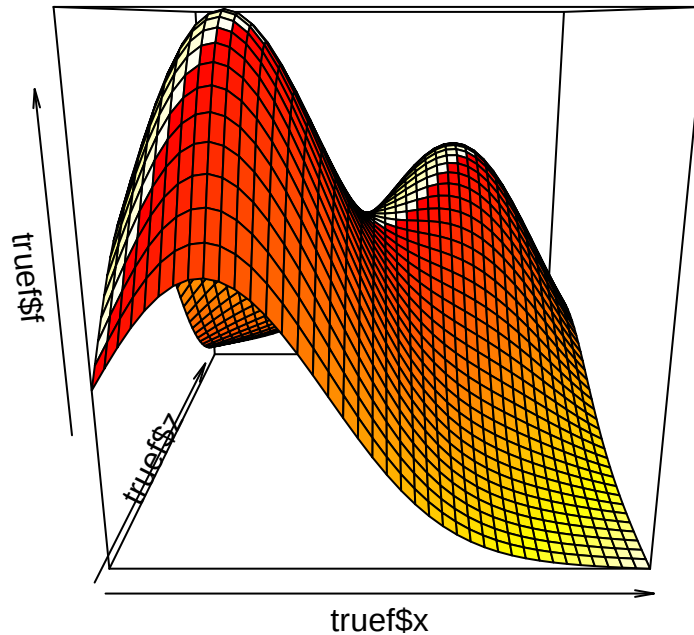
# simulate some data from a surface
n <- 5000
eg <- gamSim(2, n = n, scale = .5)

## Bivariate smoothing example
names(eg)

## [1] "data" "truth" "pr"
attach(eg)

truef <- eg$truth
persp(truef$x, truef$z, truef$f, col = heat.colors(40), main = "a. truth")
```

a. truth



To learn more about how mgcv does things look at the help files for the following functions: `smooth.terms`, `s`, `tp`s (short for `smooth.construct.tp.smooth.spec`), `te` (tensor product).

A brief summary of `s()` and its main arguments:

```
s(x, bs = "tp", fx = FALSE)
```

`fx = TRUE`: fixed d.f. regression spline

`fx = FALSE`: penalized regression spline

`bs`: - `cr`: cubic regression splines

- `cc`: cyclic cubic regression splines
- `tp`: thin plate regression spline (default)
- `te`: tensor product
- `ti`: tensor product interaction (when main effects also present)

`m` specifies the order of the derivatives in the penalty

The following code fits some simple linear models. There is a nice function to visualize fitted functions: `vis.gam`.

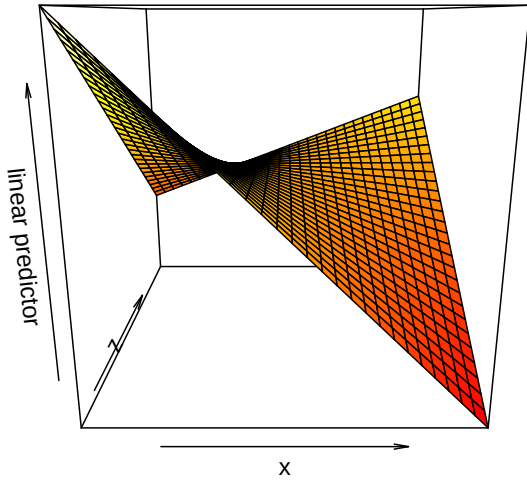
```
# linear terms plus their interaction
```

```
par(mfrow = c(1, 2))
b1 <- gam(y ~ x + z + x*z, data = data)
vis.gam(b1, main = "b. linear model")
```

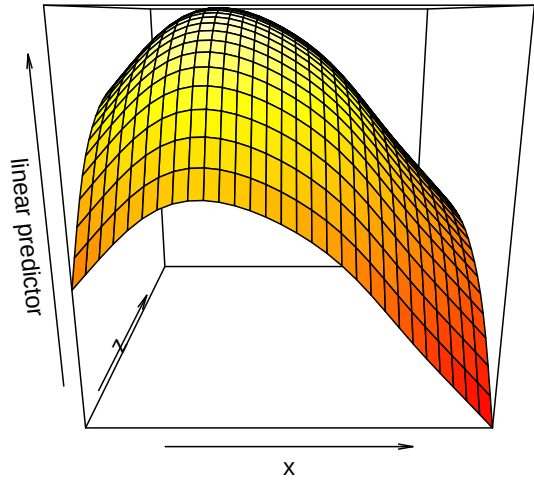
```
# additive splines
```

```
b2 <- gam(y ~ s(x) + s(z), data = data)
vis.gam(b2, main = "c. additive splines")
```

b. linear model



c. additive splines



The following code fits a two-dimensional thin-plate spline to the data. First, 200 data points are chosen at random from the n observed (x, z) points. The resulting basis is then reduced to a basis with 40 basis functions (by eigen-decomposition) and then fitted with a penalty. In the second fit, knots are chosen (not from the observed points) and the basis is not reduced to low rank before fitting, but is still penalized.

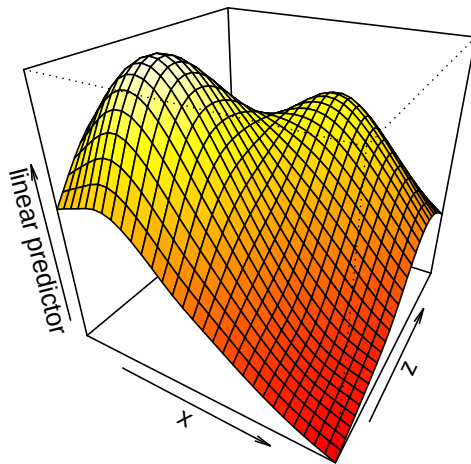
```
par(mfrow = c(1, 2))

ind <- sample(1:n, 200, replace = FALSE)
b5 <- gam(y ~ s(x, z, k = 40), data = data,
          knots = list(x = data$x[ind], z = data$z[ind]))

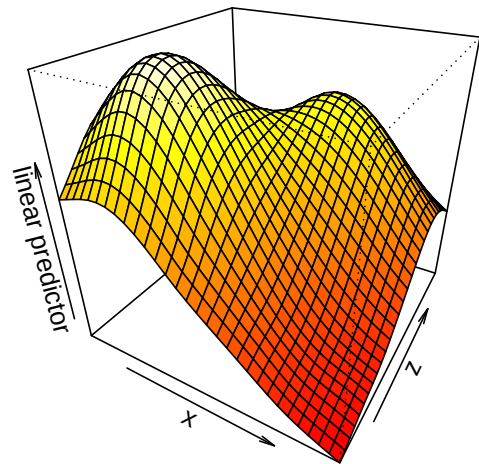
vis.gam(b5, theta = 30, phi = 30, main = "low rank thin-plate spline")

## and a pure "knot based" spline of the same data
b6 <- gam(y ~ s(x, z, k = 64), data = data,
          knots = list(x = rep((1:8-0.5)/8, 8),
                        z = rep((1:8-0.5)/8, rep(8, 8))))
vis.gam(b6, color = "heat", theta = 30, phi = 30, main = "knot-based thin-plate spline")
```

low rank thin-plate spline



knot-based thin-plate spline



```
summary(b6)
```

```
##
## Family: gaussian
## Link function: identity
##
## Formula:
## y ~ s(x, z, k = 64)
##
## Parametric coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 0.298298   0.007229  41.27  <2e-16
##
## Approximate significance of smooth terms:
##             edf Ref.df    F p-value
## s(x,z) 18.01  24.87 19.95  <2e-16
##
## R-sq.(adj) =  0.0924   Deviance explained = 9.56%
## GCV = 0.26225   Scale est. = 0.26126    n = 5000
```

```
detach(eg)
```

1. Use the `gam` function to fit a tensor product spline to the surface, with and without penalty. With and without choosing the knots. Try different values of the smoothing parameter. Different numbers of knots.
2. Repeat the above with thin-plate splines.
3. Construct your own tensor product basis and fit to the data. Plot your fitted surface. Penalties are a bit tricky (calculating the penalty matrix), so you don't have to penalize. Instead manipulate the smoothness by using more or fewer knots. You can use R functions to set up your basis, but you should fit the model using your own code (not R's fitting functions).
4. Create your own thin-plate spline basis and fit to the data. Plot your fitted surface. Again, penalties are a bit tricky (calculating the penalty matrix), so you don't have to penalize. Instead manipulate the smoothness by using more or fewer knots. Again, you should fit the model using your own code (not R's fitting functions). Set up the basis in two ways, by choosing a random number of observed data points, and by choosing a set of equally spaced knots from the domain.

You may have to write a function to evaluate the fitted spline at new covariate values.

Exercises

1. Wood Chapter 3 Exercise 4:

Show that the β minimizing

$$\|y - X\beta\|^2 + \lambda\beta'P\beta$$

is given by

$$\hat{\beta} = (X'X + \lambda P)^{-1}X'y$$

2. For P-splines (penalize squared difference between adjacent β_i values, show that

$$P = \sum_{i=1}^{k-1} (\beta_{i+1} - \beta_i)^2$$

can be written as

$$P = \beta' \begin{bmatrix} 1 & -1 & 0 & \cdot & \cdot \\ -1 & 2 & -1 & \cdot & \cdot \\ 0 & -1 & 2 & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \end{bmatrix} \beta$$

3. The ‘obvious’ way to estimate smoothing parameters is by treating smoothing parameters just like the other model parameters, β , and to choose λ to minimize the residual sum of squares for the fitted model. What estimate of λ will such an approach always produce?
4. Show that for any function f , which has a basis expansion

$$f(x) = \sum_j \beta_j b_j(x),$$

it is possible to write

$$\int f''(x)^2 dx = \beta' S \beta,$$

where the coefficient matrix S can be expressed in terms of the known basis functions b_j (assuming that these possess at least two (integrable) derivatives. As usual β is a parameter vector with β_j in its j^{th} element.